

Wise Care: AI-Powered Mental Health Access & Care Navigation

Wise Care is a full-stack care-navigation and patient preparation platform designed for the U.S. healthcare market. It acts as an administrative and preparation bridge between an individual recognizing their need for mental health support and attending their first clinical consultation.

This platform is structured to solve the entry pipeline bottleneck of U.S. healthcare by transforming raw patient intakes into actionable, non-diagnostic Care Routes and clinician-ready Care Packets, reducing waitlists, billing opacity, and outreach anxiety.

1. Problem Framing & Healthcare System Constraints

The Entry Pipeline Bottleneck

The primary barrier in U.S. mental health care is commonly assumed to be a shortage of licensed clinicians. While clinician supply is a constraint, a deeper and more immediate challenge is the high friction of the **access pipeline**. Individuals looking for care must navigate several complex systemic barriers:

1. **Search & Directory Friction:** Patients encounter outdated provider directories, opaque medical terminology (e.g., differentiating Talk Therapy, Psychiatry, Intensive Outpatient, and Peer Support), and a lack of clear starting points. The cognitive load of finding an available clinician licensed in their state frequently stalls care-seeking behavior.
2. **Financial & Insurance Opacity:** Mental health billing is complex. Patients struggle to verify in-network coverage, deductibles, copays, and sliding-scale eligibility. Fear of unexpected out-of-network bills delays care.
3. **Supply-Demand Latency (Wait Times):** The wait time for a first appointment with a licensed therapist averages 3 to 6 weeks. During this critical window, patients are left without guided support, resulting in high drop-off rates.

- 4. **Geographic & State Licensure Restraints:** U.S. healthcare regulations restrict telehealth services to states where the clinician is actively licensed. Directories rarely filter this dynamically, leading to frustrating dead ends.
- 5. **Outreach Anxiety:** Drafting emails and repeating sensitive clinical histories to intake coordinators is a massive hurdle for individuals experiencing acute anxiety or burnout.

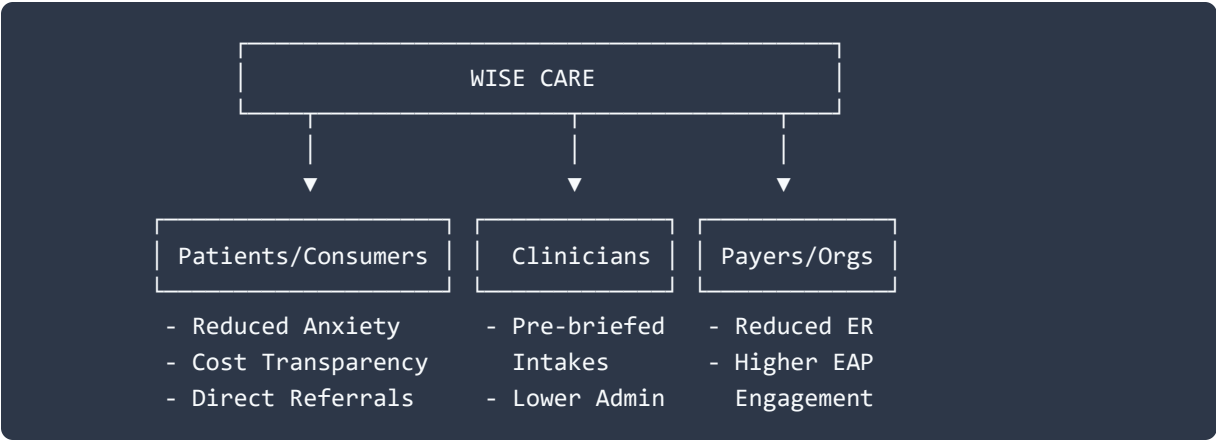
The Wise Care Solution

Wise Care operates as a pure **navigation and preparation layer** that does not diagnose, prescribe, or provide therapy. Instead, it guides the patient through:

- **Calm Structured Intake:** A quiet, non-conversational form that captures clinical and administrative details without chat fatigue.
- **Deterministic Safety Gate:** An immediate keyword-based safety scanner to route high-risk users to crisis hotlines before invoking LLMs.
- **Structured Care Routes:** Algorithmic next steps, barrier recognition, and matching criteria generated using Large Language Models with strict JSON schemas.
- **Clinician-Ready Care Packets:** Automatically compiling symptoms, history, and goals into a professional brief and outreach email, reducing intake administrative time.

2. Business Case & Value Proposition

Wise Care resolves systemic frictions for multiple stakeholders within the healthcare ecosystem:



Stakeholder Value Matrix

- **Patients:** Reduces search fatigue, demystifies insurance, and generates clear, copy-pasteable outreach messages, shortening the time-to-care.
- **Clinicians & Group Practices:** Delivers pre-prepared intake briefs (SOAP format) containing timelines, daily impact, and goals. This reduces clinician administrative burden, decreases intake drop-off, and increases slot utilization.
- **Employers & Universities:** Offers a private, structured entry point for employees/students seeking support, boosting Employee Assistance Program (EAP) utilization and reducing absenteeism.
- **Insurers & Public Payers:** Guides members to appropriate in-network care paths before symptoms escalate, reducing expensive emergency room utilization.

Ethical Monetization Policy

Wise Care enforces a strict **privacy-first data policy**. User health data, search parameters, or diagnostic briefs are never sold, shared, or monetized for advertising. Revenue is generated via:

1. **Enterprise B2B SaaS (PMPM):** Employers and universities pay a flat Per-Member-Per-Month fee to offer Wise Care as a white-labeled access portal.
 2. **Provider Integration Software:** Clinical networks pay a subscription to integrate Wise Care referrals directly into their Electronic Health Records (EHR) systems.
 3. **Payer Outcomes Pilots:** Insurers pay outcome-based fees for successful navigation pathways that redirect members from emergency care to outpatient counseling.
-

3. MVP Scoping & Strategic Decisions

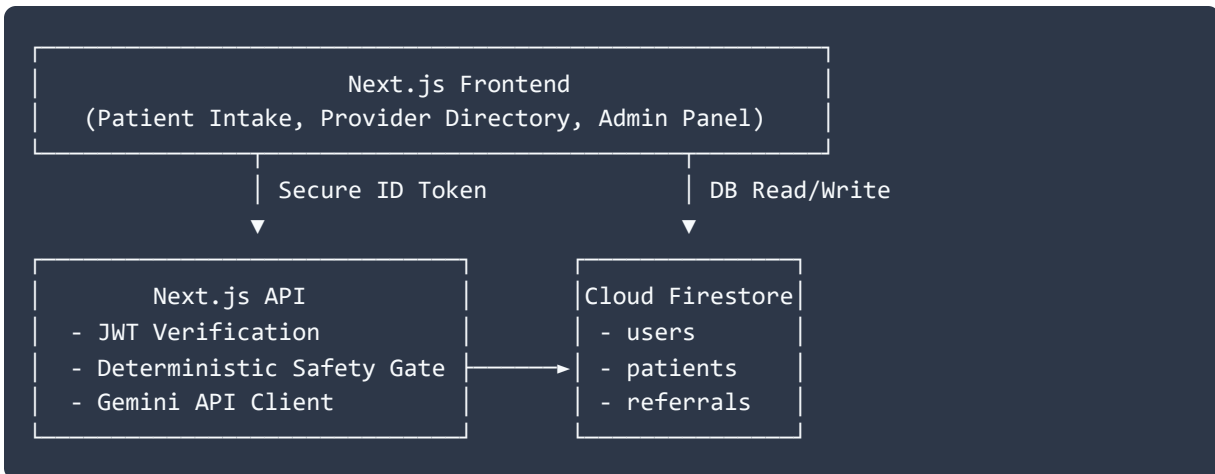
To validate the product thesis rapidly under a tight deadline, the Wise Care MVP prioritizes core workflow pathways while engineering smart fallbacks:

- **No Conversational Chatbots:** Interactive chatbots often introduce open-ended anxiety and state-tracking complexities. The MVP uses a structured multi-step form to collect metrics cleanly and predictably.

- **Developer Fallback Mode (Out-of-the-Box Execution):** If Firebase or Gemini environment variables are missing, the application automatically boots into **Developer Mode**. It utilizes browser `localStorage` for state management and local mock templates for AI generation, showing a persistent banner. This guarantees that evaluators can run the full platform flow immediately without API keys.
 - **Integrated Stakeholder Roles:** To allow testing across all interfaces (Patient, Provider, Organization, Admin) within a single browser session, the sidebar includes an interactive **Role Switcher** that changes roles dynamically.
 - **Synthetic Directory Data:** Populated with diverse provider archetypes (Private practice, psychiatric evaluation, support group, community clinic, crisis hotline) to test matching behaviors without scraping real registries.
-

4. Platform Architecture & Data Schema

The platform is built using **Next.js (App Router)**, styled with clean vanilla CSS, and uses **Firebase Authentication** and **Cloud Firestore** for data persistence.



Firestore Collection Schema

1. `users`

- **Path:** `/users/{uid}`
- **Role:** Tracks account access roles and onboarding states.

- **Schema:**

```
interface UserDoc {
  uid: string;
  email: string;
  displayName: string;
  role: 'patient' | 'solo_provider' | 'provider_org' | 'admin';
  onboardingComplete: boolean;
  createdAt: string;
  updatedAt: string;
}
```

2. **patients**

- **Path:** `/patients/{uid}`
- **Role:** Stores intake answers and active care artifacts.
- **Schema:**

```
interface PatientDoc {
  userId: string;
  displayName: string;
  intakeStatus: 'not_started' | 'started' | 'completed';
  activeCareRouteId: string | null;
  activeCarePacketId: string | null;
  activeReferralId: string | null;
  intakeAnswers: IntakeAnswers | null;
  savedProviderIds: string[];
  createdAt: string;
  updatedAt: string;
}
```

3. **careRoutes**

- **Path:** `/careRoutes/{routeId}`
- **Role:** Stores the AI-generated care pathway, barriers, and matching criteria.
- **Schema:**

```
interface CareRouteDoc {
  patientId: string;
  riskLevel: 'low' | 'moderate' | 'high' | 'crisis';
  recommendedRoute: string;
  recommendedSupportTypes: string[];
  reasoningSummary: string;
  detectedBarriers: string[];
  careGoals: string[];
  nextSteps: string[];
  matchingCriteria: {
    supportTypes: string[];
    modality: string;
    paymentPreference: string;
    urgency: string;
    state: string;
  };
  safetyMessage: string;
  isFallback: boolean;
  createdAt: string;
}
```

4. **carePackets**

- **Path:** `/carePackets/{packetId}`
- **Role:** A structured clinical brief and copyable outreach email.
- **Schema:**

```
interface CarePacketDoc {
  patientId: string;
  careRouteId: string;
  mainConcerns: string[];
  timeline: string;
  dailyLifeImpact: string[];
  careGoals: string[];
  questionsToAskProvider: string[];
  materialsToPrepare: string[];
  insurancePaymentNotes: string[];
  suggestedOutreachMessage: string;
  shareableSummary: string;
  nextStepChecklist: string[];
  isFallback: boolean;
  createdAt: string;
}
```

5. referrals

- **Path:** `/referrals/{referralId}`
- **Role:** Connects patients to providers and tracks invitation workflows.
- **Schema:**

```
interface ReferralDoc {
  patientId: string;
  patientDisplayName: string;
  providerType: 'solo_provider' | 'provider_org';
  providerId: string;
  providerName: string;
  carePacketId: string;
  careRouteId: string;
  status: 'pending' | 'accepted' | 'waitlisted' | 'declined' | 'request_info' | 'wi
  providerMessage?: string;
  createdAt: string;
  updatedAt: string;
}
```

5. Server-Side AI Workflows (Gemini API)

Wise Care integrates the `@google/genai` SDK to run server-side generative pipelines. To guarantee reliability, all Gemini calls utilize strict JSON schemas to guarantee return object types.

Core Generative Endpoints

1. **Care Route AI Workflow (`POST /api/ai/care-route`):** Processes the patient's intake form. It runs a deterministic safety check first. If clean, it invokes Gemini with `careRouteResponseSchema` to outline risks, recommended pathways, care goals, and next steps using clinical-cautious language ("may," "could," "consider").
2. **Care Packet AI Workflow (`POST /api/ai/care-packet`):** Gathers raw patient concerns, timelines, and impact metrics and converts them into a formatted PDF-ready brief (timeline, impact details, insurance notes) and a structured email draft.

3. **Follow-Up AI Workflow (`POST /api/ai/follow-up`)**: Triggered if a patient fails to connect with their matched provider. Analyzes obstacles (e.g., cost, wait times, outreach anxiety) and outputs secondary adjustments and alternative options.

Security Boundary: JWT Authentication Token Handling

To secure the backend AI endpoints from abuse, frontend fetch scripts request a Firebase ID token:

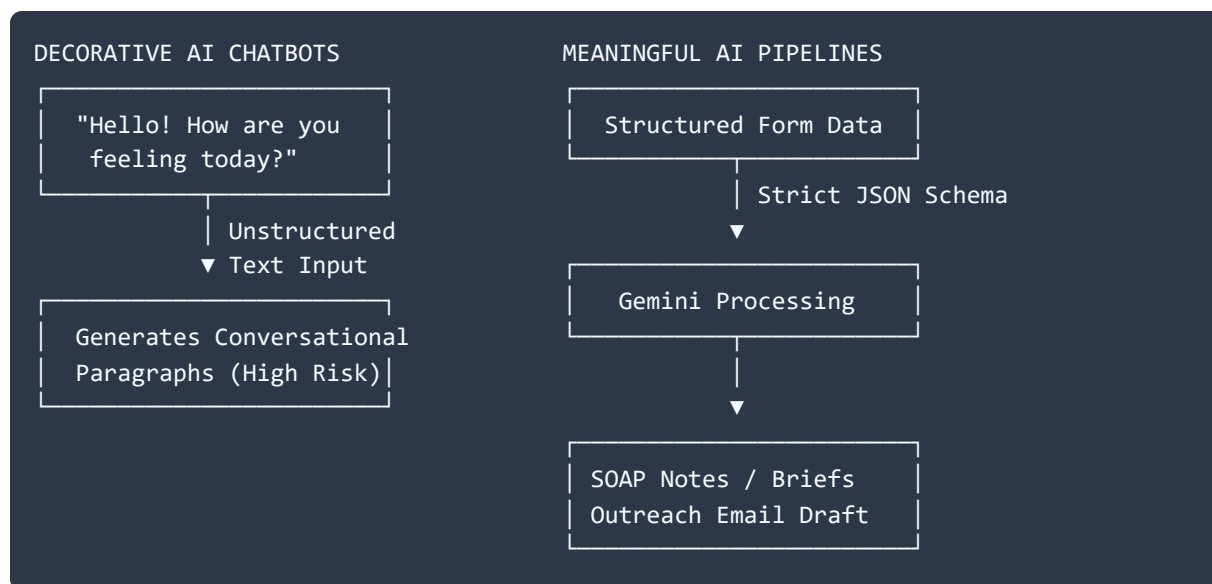
```
const token = await auth.currentUser.getIdToken(true);
```

This is transmitted in the request headers:

```
Authorization: Bearer <firebase_id_token>
```

The server-side API routes verify this Bearer token against Firebase Admin SDK boundaries before processing the AI generation.

6. Meaningful AI vs. Decorative Chatbots



A common pattern in digital health is deploying open-ended conversational chatbots (e.g., "tell me how you feel"). This design pattern fails in healthcare due to:

- **High Cognitive Load:** Patients in distress struggle to formulate detailed prompt histories.
- **Safety Risks:** Chatbots are prone to hallucinating medical advice or failing to capture hidden crisis triggers.
- **Clinician Friction:** Paragraphs of text generated by an AI chatbot do not save clinicians time during intake.

How Wise Care is Different

Wise Care uses AI as an **invisible, structured back-end utility**:

1. **Inputs are Structured:** Patients select checkboxes for symptoms, intensity, and duration, ensuring clear data boundaries.
2. **Deterministic Pre-Processing:** An algorithmic safety scanner inspects text before LLM access, guaranteeing safety.
3. **Outputs are Structured & Actionable:** Gemini is restricted to returning JSON schema objects. Instead of generating conversational filler, it builds copyable outreach templates and SOAP summaries, solving real administrative workflows.

7. 30-Day Production Roadmap

To transition this prototype into a production-ready, HIPAA-compliant system, we propose the following 30-day engineering plan:

30-DAY ROADMAP			
WEEK 1	WEEK 2	WEEK 3	WEEK 4
HIPAA DB & NextAuth.js	NPI API Verification & Google Maps Geolocation	Candid Health API Insurance Checks	Twilio SMS & EHR Connect

Week 1: Secure Infrastructure, Auth & NPI Verification

- **HIPAA-Compliant Database:** Migrate from standard Firestore to an encrypted database hosted on a HIPAA-compliant cloud (e.g., AWS RDS with KMS encryption or Aptible).
- **NextAuth.js MFA:** Implement secure multi-factor authentication (MFA) for providers.
- **N NPPES Registry API Integration:** Query the National Provider Identifier (NPI) Registry API during provider sign-up to verify licenses and active clinical standing automatically.

Week 2: Directory Integration & Geolocation Mapping

- **Real Directories:** Integrate SAMHSA and OpenPath APIs to feed real outpatient clinic data.
- **Google Maps API:** Add postal code radius sorting for patients seeking in-person care.
- **State Licensure Scraping:** Build automated verification check scripts targeting state licensing boards.

Week 3: Real-Time Insurance & Benefit Verification

- **Eligibility Clearinghouse integration:** Connect to **Candid Health** or **Change Healthcare** APIs to verify insurance benefits, remaining deductibles, and co-pay amounts instantly.
- **Sliding Scale Portal:** Implement secure document uploading (using encrypted Cloud Storage) to automate sliding-scale category verification.

Week 4: Multi-Channel Communications & EHR Integration

- **Secure Email:** Set up HIPAA-compliant secure email servers (e.g., LuxSci, Mailgun HIPAA) to transmit outreach summaries.
 - **Direct EHR Integrations:** Implement API endpoints to sync care packets directly into common EHR platforms (e.g., Athenahealth, SimplePractice).
 - **Twilio SMS Alerts:** Add automated patient follow-up checks via SMS using a HIPAA-compliant Twilio account.
-

8. Safety, Guardrails & Responsibility

Three-Layer Safety Guard

- **Layer A (Deterministic Regex Scanner):** If the intake contains keywords like `suicide`, `self-harm`, `harm myself`, or `overdose`, the application immediately skips the LLM and serves a hardcoded **Crisis Route** pointing to 988.
 - **Layer B (Prompt-Level Rules):** System instructions forbid Gemini from diagnosing conditions, recommending specific drugs, or offering false comfort (*"you are safe"*). It mandates tentative phrasing (*"may suggest," "might consider"*).
 - **Layer C (Clinical Responsibility):** Clear scoping ensures patients understand that the tool acts as a navigation layout rather than a therapy substitute.
-

9. Setup, Execution & Interactive Testing

Prerequisites

- Node.js 20 or later

Installation

1. Install dependencies:

```
npm install
```

2. Set up environment variables by copying `.env.example`:

```
cp .env.example .env.local
```

Provide your values in `.env.local`:

```
# Live Gemini API Key configuration
GEMINI_API_KEY=AIzaSy...
GEMINI_MODEL=gemini-2.5-flash

# Firebase configuration parameters
NEXT_PUBLIC_FIREBASE_API_KEY=your-api-key
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=your-auth-domain
NEXT_PUBLIC_FIREBASE_PROJECT_ID=your-project-id
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=your-storage-bucket
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=your-sender-id
NEXT_PUBLIC_FIREBASE_APP_ID=your-app-id
```

3. Deploy Firestore Security Rules:

```
firebase deploy --only firestore:rules
```

4. Start the local dev server:

```
npm run dev
```

Open <http://localhost:3000> in your browser.

Simulated Developer Accounts

In Developer Mode (missing environment configs), log in using these simulated emails (no password required):

- **Patient:** `patient.demo@wisecare.test`
- **Solo Clinician:** `clinician.demo@wisecare.test`
- **Clinic Org:** `clinic.demo@wisecare.test`
- **Admin:** `admin.demo@wisecare.test`

To configure a Platform Admin in Firebase Mode, register a user account normal, locate their UID in the Firestore Console, and change their `role` field value in the `users` collection to `"admin"`.